

Network Programming in Haskell

Haskell, named after logician Haskell Curry, is a standardised, general-purpose, purely functional programming language, with non-strict semantics and strong static typing. This eleventh article on Haskell covers network programming.



Let us begin with a simple TCP (transmission control protocol) client and server example. The *network* package provides a high-level interface for communication. You can install the same in Fedora, for example, using the following command:

```
$ sudo yum install ghc-network
```

Consider the following simple TCP client code:

```
-- tcp-client.hs

import Network
import System.IO

main :: IO ()
main = withSocketsDo $ do
    handle <- connectTo "localhost" (PortNumber 3001)
    hPutStr handle "Hello, world!"
```

```
hClose handle
```

After importing the required libraries, the *main* function connects to a localhost server running on Port 3001, sends a string 'Hello, world!' and closes the connection.

The *connectTo* function defined in the *Network* module accepts a hostname, port number and returns a handle that can be used to transfer or receive data.

The type signatures of the *withSocketsDo* and *connectTo* functions are as follows:

```
ghci> :t withSocketsDo
withSocketsDo :: IO a -> IO a

ghci> :t connectTo
connectTo :: HostName -> PortID -> IO GHC.IO.Handle.Types.
Handle
```

The simple TCP server code is illustrated below: